Data Structures and Algorithm

COURSE CODE: CS-201

TOPIC: SINGLY LINKED LIST INSERTION

COURSE INSTRUCTOR: HABIBA ARSHAD

CLO Covered in this lecture

CLO2: **Apply** and design various data structures methods for specified applications.

Goals for today

To study:

- Singly Linked List
  - Insertion
  - Deletion

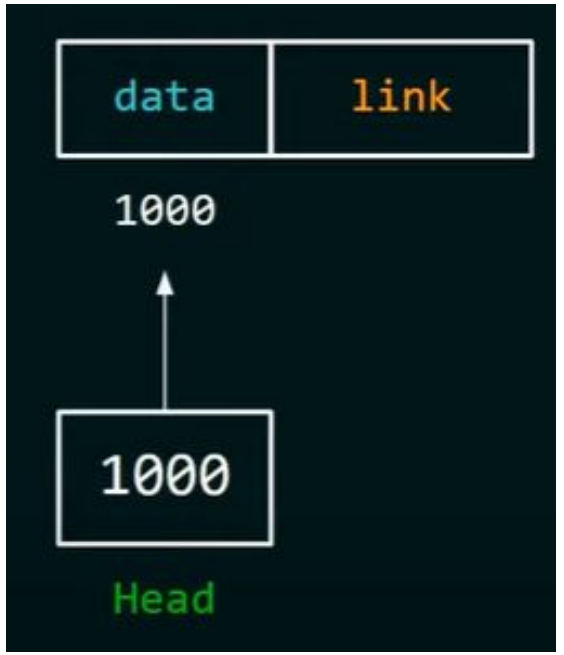
Creation of node

- To create a linked list in the memory, use structure which is self referential structure.
- In self referential structure one member of the structure is a pointer that points to the structure itself


```
struct node
        {
          int data; // any data type can be used
                  struct node *link; // this points to the structure itself
        };
```

Creation and insertion of data in node

```
int main()
{
    struct node *head = NULL;
    head = (struct node*)malloc(sizeof(struct node));
    head->data = data;
    head->link = NULL;
}
```



Insertion of an element into the linked list at various positions:

Insertion of node into a linked list requires a free node in which the information can be inserted and then the node can be inserted into the linked list.

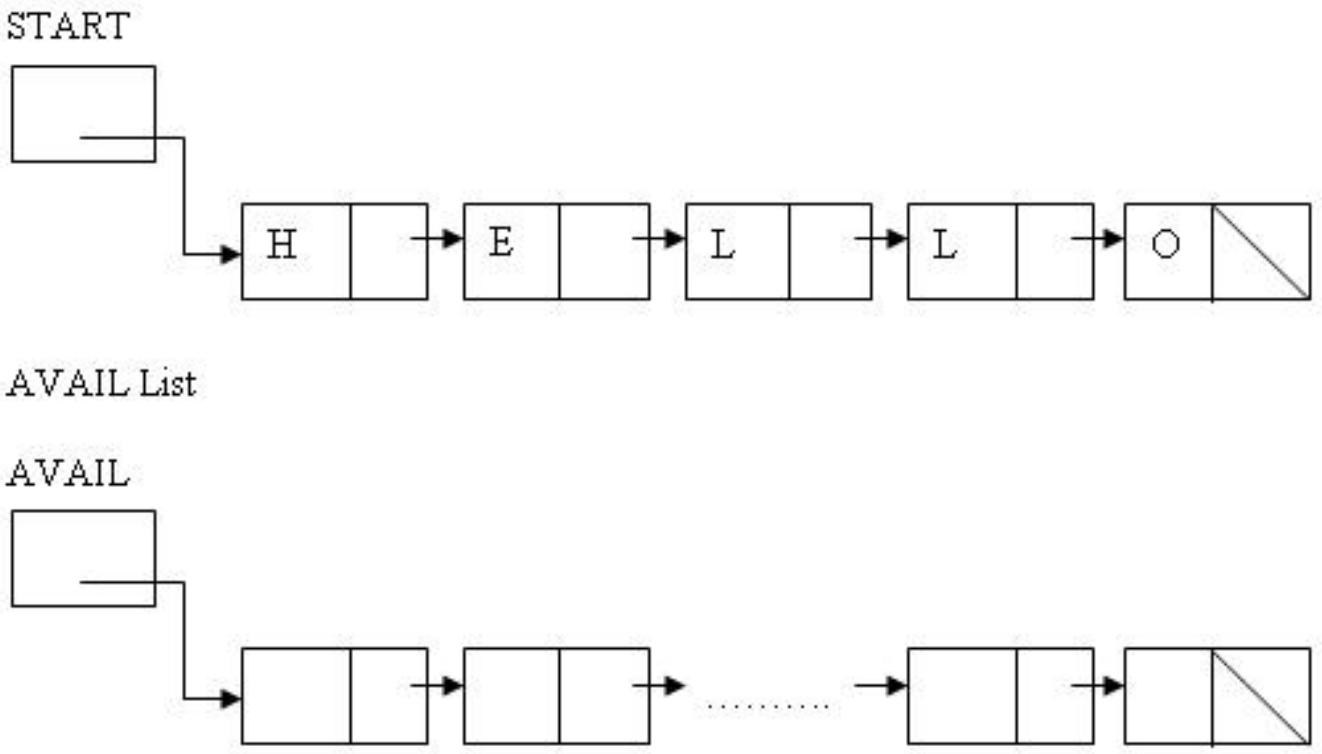
- Before header or at beginning.
- At the end of list.
- At the specified position.

Availability List or AVAIL List.

It is a list of free nodes in the memory. It contains unused memory cells and these memory cells can be used in future. It is also known as ‘List of Available Space’ or ‘Free Storage List’ or ‘Free Pool’.

It is used along with linked list. Whenever a new node is to be inserted in the linked list, a free node is taken from the AVAIL List and is inserted in the linked list.

Similarly, whenever a node is deleted from the linked list, it is inserted in the AVAIL List. So that it can be used in future.


START
H
E
L
L
O
AVAIL List
AVAIL

Overflow and Underflow

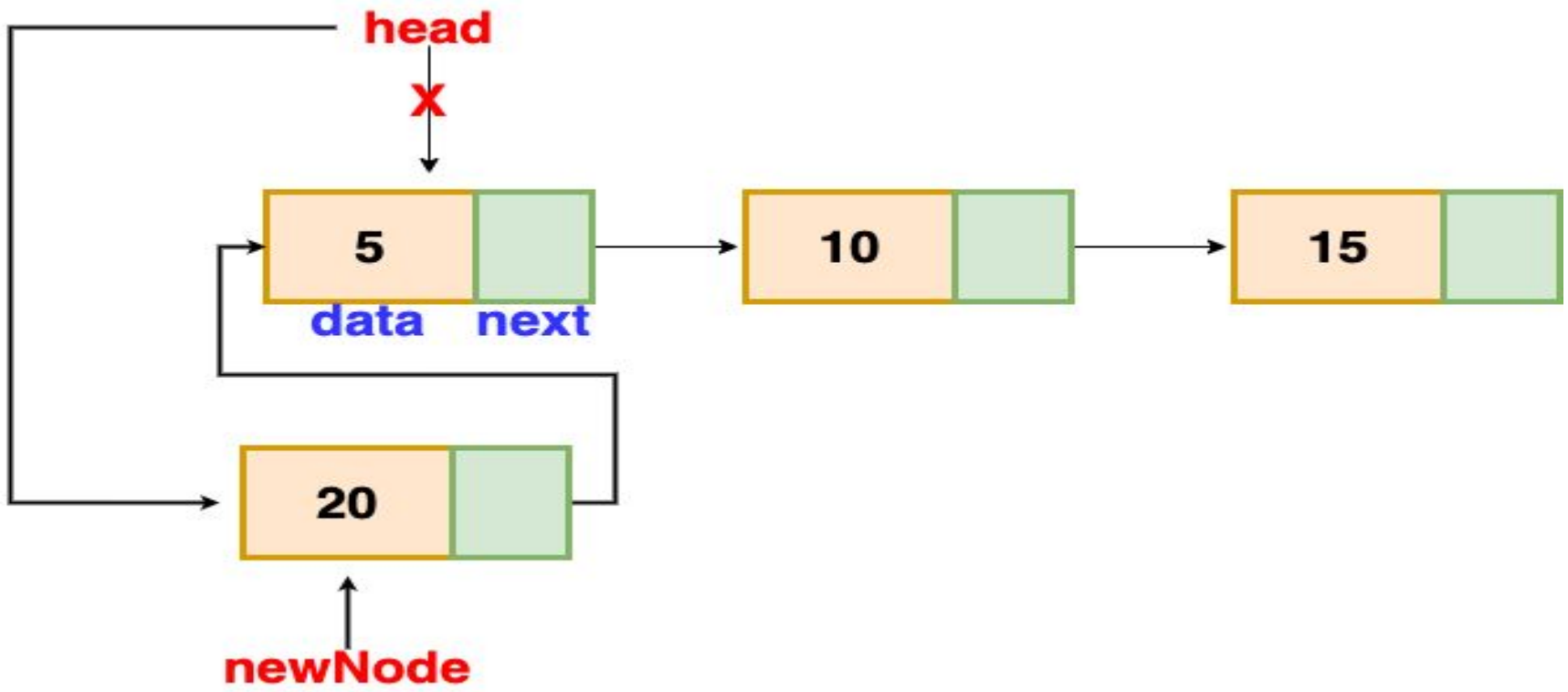
Overflow

Overflow happens at the time of insertion. If we have to insert new data into the data structure, but there is no free space i.e. availability list is empty, then this situation is called 'Overflow'.
Overflow will occur with our linked list when AVAIL = NULL and there is an insertion

Underflow

Refers to the situation where **one wants to delete data** from a data structure that is empty.
The situation is handled by printing the message UNDERFLOW
Observe that underflow will occur when our linked lists when START = NULL and there is a deletion

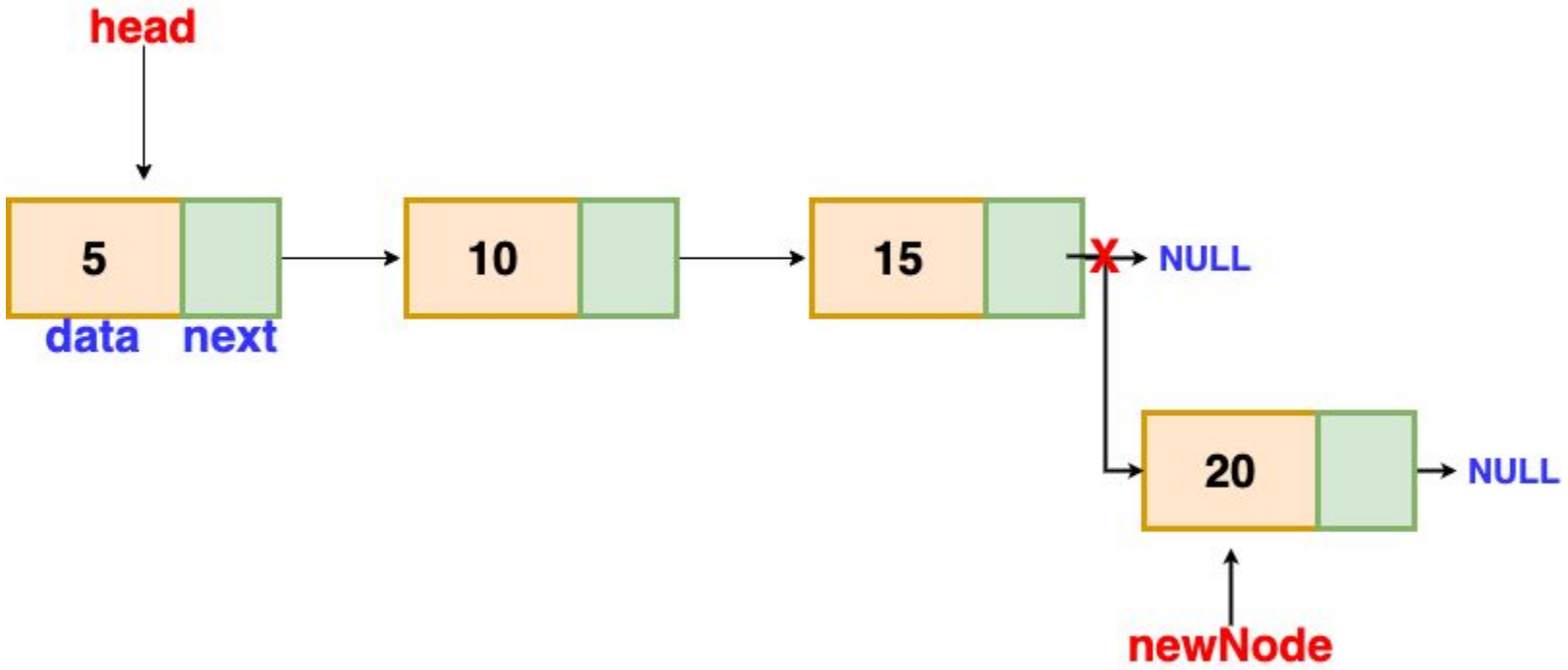
Insertion at beginning



Insertion at beginning

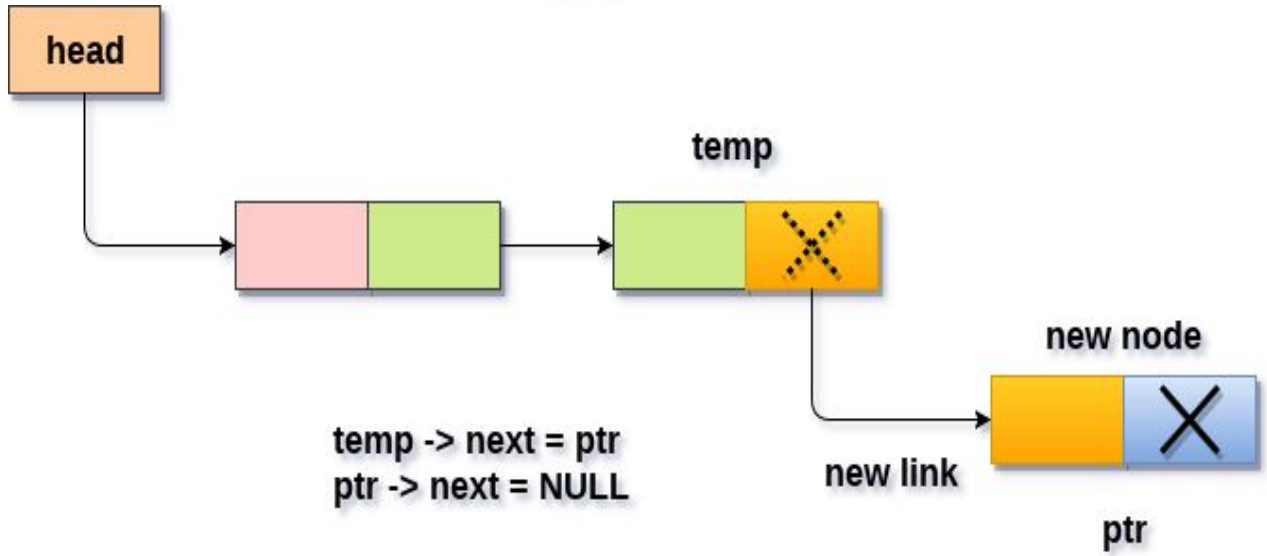
```
struct node *temp;
temp = (struct node *)malloc(sizeof(struct node));
temp->info=data;
temp->link=start;
start = temp;
```

Insertion At end



Insertion at End

```
struct node *p, *temp;
temp = (struct node *)malloc(sizeof(struct node));
temp->info=data;
p = start;
while(p->link != NULL){
    p = p->link;
}
p->link = temp;
temp->link=NULL;
```



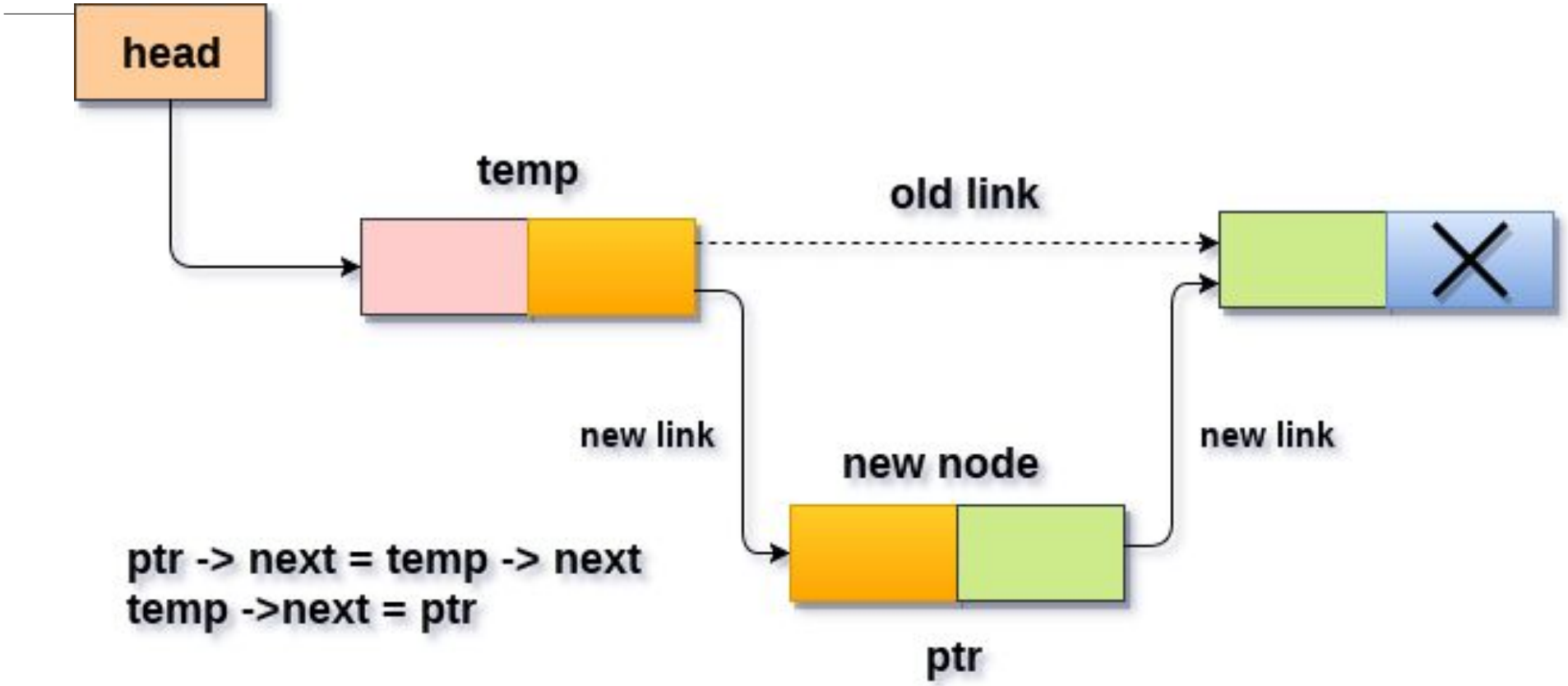
Inserting node at the last into a non-empty list

OPERATION ON SINGLE LINKED LIST

Insertion at the specified position

- To insert a new node at the specified position we have to search position in the list.
  - Then we can insert the node after the position. Let the address of the specified node be kept in ptr.
- Adjust the pointer so that next pointer of the specified node pointed to the ptr.
- The next pointer of the specified node will point to the next pointer of the node pointed by new ptr.

Insertion at the specified position



position

```
int i;
struct node *p, *temp;
temp = (struct node *)malloc(sizeof(struct node));
temp->info=data;
if(pos==1){
    temp->link = start;
    start = temp;
    return start;
}
p = start;
for (i=1; i<pos-1 && p != NULL; i++)
{
    p= p->link;
}
if(p == NULL)
{
    cout<<"There are less than"<<pos<<"elements."<<endl;
}
else
{
    temp->link = p->link;
    p->link=temp;
}
```

Questions?